

理解 A2A 與 MCP 協議

<https://gemini.google.com/app/fa9d98245f44bbd0>

User prompt: 聽說有多AGENT的A2A協議，請介紹

Response: **Agent-to-Agent (A2A)** 协议是一种专门为 AI 智能体 (AI Agents) 之间进行跨系统、跨框架协作而设计的开放标准通信协议。由 Google 等公司倡导发起，目前托管于 Linux 基金会 (Linux Foundation)。

如果说 **MCP (Model Context Protocol)** 解决了“AI 智能体如何调用工具和访问数据”的问题，那么 **A2A** 解决的就是“AI 智能体之间如何进行对话与协同分工”的问题。

核心痛点：为什么需要 A2A？

传统的 Multi-Agent 框架（如 LangGraph、CrewAI 或 AutoGen）通常只能在同一个生态内部实现多 Agent 协同。一旦遇到以下场景，就会面临跨系统壁垒：

1. **跨框架/跨语言**：用 Python (LangGraph) 写的 Agent 无法方便地与 Java 或 Go 编写的 Agent 直接交互。
2. **跨公司/跨安全域**：企业 A 的智能体（如供应商系统）需要与企业 B 的智能体（如采购系统）协作，但双方无法共享内部代码、Memory 或 Tool 实现。

A2A 提供了统一的“跨系统通用语言”，将各个 Agent 视为**不透明 (Opaque)** 的独立实体，在不暴露内部专有逻辑的前提下实现安全协作。

A2A 的核心架构组件

A2A 采用了基于 **JSON-RPC 2.0 over HTTPS** 的 C/S（客户端-服务器）架构：

- **Agent Card (智能体名片)**
 - 每个兼容 A2A 的 Agent 都公开一个 JSON 格式的“名片”(类似于 LLM 的 Model Card)。
 - 名片中包含了 Agent 的名称、功能描述、支持的数据类型、端点 URL 以及身份验证要求（支持 API Key、OAuth 2.0 等），用于 Agent 之间的自动**能力发现 (Discovery)**。
- **A2A Client & A2A Server**
 - **Client (发起方)**：发起任务请求的 Agent。
 - **Server (接收方)**：暴露 HTTP 端点、接收并处理任务的远程 Agent。
- **Task Management (任务生命周期)**

- 将复杂的协作拆解为“Task”，并定义了清晰的状态流转（如 Submitted -> Working -> Input Required -> Completed / Failed）。
- **Artifacts & Parts（工件与数据块）**
 - 支持文本（TextPart）、文件（FilePart）及结构化 JSON 数据(DataPart) 的交换，并支持生成结果（如报告、图片等 Artifacts）的增量流式传输（基于 SSE）。

典型工作流程（三步走）

1. **发现 (Discovery)**：Agent A 收到任务后，查找外部 Agent B 的 Agent Card，确认其是否具备解决该问题所需的技能。
2. **鉴权 (Authentication)**：Agent A 根据 Agent Card 的要求，通过 OAuth 2.0 / API Key 等安全方式对 Agent B 进行身份校验和授权。
3. **任务协作 (Communication)**：
 - 短任务：通过同步 Request/Response 完成。
 - 长时间任务（耗时几小时/几天）：通过 HTTP SSE（Server-Sent Events）实现实时状态流式更新，或通过 Webhook 进行异步推送到 Client。

A2A vs. MCP：二者有何不同？

维度	MCP (Model Context Protocol)	A2A (Agent2Agent Protocol)
主导方	Anthropic	Google / Linux Foundation
交互对象	Agent ↔ 外部工具/数据源（如数据库、API、文件系统）	Agent ↔ Agent（智能体对等协作）
定位	工具与上下文接入标准	多 Agent 跨系统通信协议
典型示例	客服 Agent 使用 MCP 连接内部 CRM 数据库查询订单。	客服 Agent 发现商品缺货，通过 A2A 向供应商 Agent 发起采购订单。

补充：在实际落地场景中，MCP 与 A2A 是互补关系。通常一个系统内部使用 MCP 连接基础设施与数据，外部通过 A2A 与其他跨界 Agent 进行协商交互。

Sources

1. [Agent-to-Agent \(A2A\) 协议全面解析：定义、原理、应用与未来](#)
2. [What is A2A protocol \(Agent2Agent\)? - IBM](#)

3. 什么是A2A 协议(Agent2Agent)？ - IBM

4. Agent2Agent (A2A) 协议资源

User prompt: A2A怎麼解決多智能體P2P通信時出現的信息損耗問題。

Response: 在多智能體（Multi-Agent）點對點（P2P）通信中，最棘手的問題之一就是「信息損耗」（Information Decay/Loss）與「上下文漂移」（Context Drift）。

當 Agent A 把任務傳給 Agent B，Agent B 再轉交給 Agent C 時，如果只靠自然語言（Text）來回傳遞，訊息很容易像「傳話遊戲」一樣，丟失原始指令細節、丟失中間狀態，甚至產生幻覺。

Agent-to-Agent (A2A) 協議的核心設計理念是「黑盒解耦（Opacity by Design）」——各 Agent 互不共享內部 Memory 和 Prompt。為了在不暴露內部狀態的前提下解決信息損耗，A2A 協議在架構設計上採取了以下關鍵機制：

1. 「任務（Task）與工件（Artifact）」的不可變分離

傳統多 Agent 系統常把「過程討論」與「最終結果」混在一起傳遞，導致信息越傳越雜。A2A 协议在数据模型上做了严格区分：

- **Message（訊息）**：用於動態對話、補充提問、狀態確認（屬於易失的過程數據）。
- **Artifact（工件）**：任務執行的產出結果（如生成的 PDF 報告、結構化 JSON 數據等）。
Artifact 在 A2A 中被定義為「不可變（Immutable）」的實體。
- **解決損耗方式**：中間 Agent 傳遞的是原始 Artifact 的引用或完整 Payload，而非自己二次摘要（Re-summarize）後的文本，徹底杜絕了二次 LLM 加工帶來的幻覺和信息衰減。

2. 強型別的 DataPart 結構化約束

A2A 支持多模態傳輸（Part 機制），包含 TextPart、FilePart 與 DataPart。

- 為了防止自然語言解析不一致，A2A 鼓勵使用 **DataPart** 來傳輸嚴格符合 JSON Schema 的結構化數據。
- **內容類型協商（ContentType Negotiation）**：Agent Card 在名片中就已宣告其支持的 Schema 和 ContentType。如果接收端不理解特定的 Schema，A2A 會在邊界處直接拋出 `ContentTypeNotSupportedError`，而不是強行用 LLM 模糊理解，從源頭阻止無效信息的傳播。

3. 基於 contextId 與 taskId 的狀態持久化

在分散式 P2P 網絡中，網路抖動或長任務中斷是造成 context 丟失的主因。

- A2A 為每個任務分配全局唯一的 `taskId`，並允許跨多個 Task 共享一個 `contextId`（用於錨定同一個 Session/工作流）。
- **斷線重連機制**：如果 P2P 網絡連接中斷，Client 端不需要重新發送整段歷史（這會引發 Context 冗餘或損耗），只需調用 `tasks/get` 或重新訂閱 SSE 流（`message/stream`），Server 端會重新播放完整的 Task State Snapshot（狀態快照）。**真理（Truth）始終存在於 Task 的持久化歷史中，而非傳輸管道中。**

4. 顯示的狀態機與「明確求助（Input Required）」

自然語言對話常因為「意圖模糊」導致 Agent 瞎猜。A2A 引入了標準的 8 階段 Task 狀態機（`submitted -> working -> input-required -> completed` 等）：

- 當下游 Agent 發現上游傳來的 Prompt/參數缺少關鍵細節時，它不會硬著頭皮生成（避免產生錯誤信息），而是主動將 Task 切換為 `input-required` 狀態。
- 這會觸發一個結構化的反饋請求，要求上游 Agent 或 `human-in-the-loop` 精準補充缺失的信息，確保傳播鏈條上的每一個數據點都是明確的。

5. Capabilities 的精確宣告（Agent Card）

信息損耗往往發生在「找錯 Agent 辦錯事」。

- A2A 的 Agent Card（`/.well-known/agent.json`）明確界定了 Agent 的能力邊界、輸入/輸出格式及 Auth 要求。
- 發起方 Agent 在分發任務前，能以機讀（Machine-readable）的方式確定目標 Agent 是否能**完全理解**該任務。如果不匹配，在路由階段就予以拒絕，避免了將任務丟給不合適的 Agent 導致的理解偏差。

總結

傳統 P2P 多 Agent 通信	A2A 協議解決方案
純自然語言口耳相傳（越傳越偏）	不可變 Artifact + 結構化 DataPart（保持原汁原味）
依賴網路 Session 記憶（斷線即忘）	持久化 taskId + Snapshot 重放（數據不丟失）
資訊不足時盲目猜測（產生幻覺）	input-required 狀態阻斷（精準補全上下文）
模糊的自然語言能力描述	強型別 Agent Card Schema 協商

通过这些协议级的规范，A2A 把原本不可控的“LLM 自由對話”包裝成了具有強約束、可追溯、狀態明確的分布式微服務調用，從而在根源上消解了信息損耗。

Sources

1. Model Context Protocols (MCP) and Agent-to-Agent (A2A ... - Medium
2. Understanding A2A — The protocol for agent collaboration

User prompt: 與這些規則比較一下。# 开发任何 AI 应用的通用防翻车原则与检查清单 > 适用对象: 所有基于大模型 (LLM / VLM / Agent) 构建的应用——RAG、单 Agent、多 Agent、自动化流水线均适用。> 核心共识只有一条: **LLM 输出不可信, 所有关键决策必须由确定性代码 (deterministic gate) 把关。** > 本原则由「多 Agent 防翻车架构」提炼泛化而来; 底层全部对应经典软件工程范式 (Single Source of Truth、Zero-Trust 边界、Reconciliation、Circuit Breaker、Observability)。## 思维导图 ``mermaid mindmap root((AI应用<br>防翻车原则)) 单一可信源 SSOT 思想 LLM只作提案者 多Agent改共享状态 边界校验 最小权限工具 Schema校验 白名单接口 分层过滤网 格式维 事实维(可溯源) 逻辑维(勾稽) 交付对账 反向核对 无来源即删/存疑 熔断降级 超时强杀 优雅降级 可观测 质量分 异常率/隔离率 人工兜底 自动优先 决策沉淀KB `` --- ## 一、七大通用原则 ### 原则 1 · 单一可信源 (Single Source of Truth) 不让 LLM 持有或成为关键数据的"真相"。凡可确定性获取的数据 (解析、数据库、规则计算), 一律以确定性层为主源; LLM 只作**提案者/补齐者**, 永不作主源。 - 对应范式: SSOT。多 Agent 场景进一步要求"废除点对点私聊, 改走结构化共享状态"。 - 反模式: 把 Agent 之间传递的纸条当成事实、让模型直接输出最终数字。 **示例** -  财报分析: 数值以 pdfplumber 确定性解析层为主源, LLM 只补缺失科目; LLM 与确定性值分歧 >5% 时一律留确定性主值。 -  多 Agent: 引入"医院病例本"式共享状态, 检索角色写`风险原始数据`字段、建模角色写`模型计算结果`字段, 互不覆写, 而非互相发消息。 -  让 LLM 直接输出"总负债 100 亿"作为最终入库值, 下游 Agent 当成铁律层层推导, 偏差指数级放大。 ### 原则 2 · 边界校验 (Role Boundary / Zero-Trust Input) 任何进入系统的外部输入 (含 LLM 输出、用户上传、工具返回) 都必须过**结构化校验**: Schema 约束 + 类型/范围检查 + 白名单。 - 对应范式: 最小权限 + 输入验证。Agent 的工具集应**最小必要** (只读工具 + 受闸门保护的写工具, 不给 Bash/Write 绕过闸门)。 - 反模式: 直接 `eval`/信任 LLM 返回的 JSON、允许 Agent 任意写文件或调外部接口。 **示例** -  resolver agent 只注册 `bs\_candidates()` (只读)、`report\_unit()` (只读)、`submit\_spec()` (受 `A≈L+E` 闸门保护) 等最小工具, **不给 Bash/Write**, 使其无法绕过勾稽闸门直写数据库。 -  RAG 应用: 对用户 query 与检索结果做 Schema 校验, 非法字段直接拒绝, 不进 LLM。 -  `json.loads(llm\_output)` 后直接 `exec` 执行, 或允许 Agent 调 `os.system` 任意命令。 ### 原则 3 · 分层过滤网 (Layered Validation) 对模型产物做**三维**校验, 任一不过即拦截/降级: 1. **格式维**: 结构是否符合代码定义的数据模型; 2. **事实维**: 关键结论能否在前述可信源中找到溯源引用 (span / 出处); 3. **逻辑维**: 与已确认数据是否自相矛盾 (勾稽、跨字段业务规则)。 - 对应范式: Validation by deterministic calculators, LLM 不参与裁决。 **示例** - 格式维: LLM 返回的结构不是合法 JSON / 缺必填字段 → 拒绝。 - 事实维: 模型说"该企业负债 100 亿", 但在可信源 (解析层) 里找不到对应单元格与页码 → 判为无来源幻觉。 - 逻辑维: 资产 ≠ 负债 + 权益、或"有息负债 > 总资产" → 勾稽不过, 隔离进审查队列。 - 反模式: 用"模型很自信"或"概率最高"来代替代码校验。 ### 原则 4 · 交付前对账 (Reconciliation) 在产物交付给用户/下游前, 反向核对: 报告里的每句结论、每个数据点, 都能在可信源中找到对应字段; 无中生有者强制删除或打"存疑"标签。 - 宁可产物因剔除数据而简短, 也绝不交付自相矛盾的带病产

物。 **示例** - ☒ 信用备忘录生成前，对账程序逐条核对"报告提到的违约风险"是否能在`风险`字段找到依据；找不到则强制删除或贴上"数据存疑"标签。 - ☒ 客服 AI 给退款结论前，反向核对订单状态（可信源），禁止模型凭"我记得你是VIP"这类无来源说法放款。 - ☒ 模型在报告里写"总负债已突破百亿"，无人对账直接发给客户，事后发现是幻觉。 ### 原则 5 · 熔断与优雅降级 (Circuit Breaker + Graceful Degradation) 任何外部/模型调用都要有**计时器**与**失败上限**：超时或连续失败即熔断、释放资源，绝不拖垮整体；非核心节点填默认值或跳过，核心节点升级人工/告警。 - 对应范式：Circuit Breaker (《Release It!》)。弱模型/慢接口必须能被硬上限强杀（子进程 kill，而非软超时）。 **示例** - ☒ 单个报告解析设`RESOLVER\_HARD\_CAP=600s`，在独立子进程跑 agent，超时`subprocess.kill()`强杀，避免弱模型把本地 GPU/API 占满、拖垮其他任务。 - ☒ LLM 调用设`LLM\_WALLCLOCK=45min`上限 + 每处`try/except`兜底，失败返回确定性默认值而非崩溃。 - ☒ 扫描件无文本层且`HAS\_VLM=False`时 fail-fast，拒绝静默产出残缺结果。 - ☒ 一个卡死的检索请求无限等待，把整个服务的并发线程全部占满，全线超时。 ### 原则 6 · 可观测 (Observability) 每个执行节点的消耗 (token / 耗时 / 成功率 / 质量分)、异常率、隔离率必须**实时量化**。低质要可测、可监控，而非事后翻案。 - 没有看板，就等于盲飞。 **示例** - ☒ 看板实时展示：每个 Agent / 每一步的 token 消耗、耗时(ms)、工具调用成功率；批量入库后的"隔离率 / 覆盖率 / grounding 率"报告。 - ☒ 发现某负责日期格式化的 Agent 竟在调昂贵顶级大模型、日烧几百块且失败率 10% → 立刻换成写死的正则表达式。 - ☒ 等客户投诉"数据错了"才去翻几万行日志找问题。 ### 原则 7 · 人工最后兜底 (Human-in-the-loop) 自动优先，但**唯一无法自动化**的点（模型穷尽尝试仍失败）由人收尾；人工决策留痕、沉淀为可复用知识 (specs / KB)，避免反复重试与自动绕过关键闸门。 **示例** - ☒ 长尾报告解析：agent 自动修正失败 (`max\_iterations` 用尽或超时) → 升级人工复核台，业务人员三选一（确认草稿 / 点选正确行 / 标记豁免），所有决策写审计日志。 - ☒ 人工修正沉淀为`specs/`与 KB，下次同主体入库自动复用，越早修越省人工。 - ☒ 用`--signoff`批量自动放行所有 critical 隔离项，等于关掉质量闸门，污染全局知识库。 --- ## 二、三条记忆口诀 (三原则法则) | 口诀 | 含义 | |-----|-----| | **稳信息一致** | 单一可信源 + 严格边界，杜绝"传纸条式"信息失真 | | **稳任务流转** | 依赖图校验防死锁 (多 Agent) / 超时熔断防卡死 (单流程) | | **稳幻觉不扩散** | 出口过滤 + 交付对账，把错误锁在局部，绝不流向下流 | --- ## 三、落地检查清单 (Checklist) > 每个特性上线前逐条过。 ☒ = 已落实， ☒ = 缺失风险点。 ### A. 数据主权与可信源 - [] 关键数据是否有**确定性主源** (解析/DB/规则计算)，而非依赖 LLM 生成？ - [] LLM 是否被限定为"提案/补齐"角色，且**冲突时以确定性值为准**？ - [] 多 Agent 场景下，是否存在**结构化共享状态** (非点对点私聊)？ - [] 每条结论是否**强制带溯源引用** (page / 坐标 / 字段 id)？ ### B. 边界与权限 - [] 所有外部/模型输入是否过**Schema + 类型/范围**校验？ - [] Agent 工具集是否**最小必要** (只读 + 受闸门写，无 Bash/Write 绕过)？ - [] 是否**白名单**限制工具/接口调用范围？ - [] 写操作是否必须经过**确定性闸门**才落盘 (不可被模型直写绕过)？ ### C. 校验过滤网 - [] 是否实现**格式维**校验 (结构合法性)？ - [] 是否实现**事实维**校验 (关键值可溯源到可信源)？ - [] 是否实现**逻辑维**校验 (跨字段勾稽 / 业务规则，如`A≈L+E`)？ - [] 校验是否**100% 确定性代码**完成，LLM 不参与裁决？ ### D. 交付对账 - [] 产物交付前是否做**反向对账** (结论 ↔ 可信源字段)？ - [] 无来源 / 自相矛盾项是否**强制删除或标注存疑**，而非静默保留？ - [] 是否接受"产物因剔除数据变简短"而不交付带病结果？ ### E. 韧性 (熔断/降级) - [] 每个外部/模型调用是否有**超时计时器**？ - [] 是否有**重试上限 / 失败熔断**，避免无限重试与资源占满？ - [] 弱模型/慢接口是否有**硬上限强杀**机制 (而非仅软超时)？ - [] 非核心失败是否**填默认值/跳过**保持主干通行？ - [] 核心失败

是否**升级告警/人工**，而非静默降级？#### F. 可观测 - [] 是否有**看板**实时展示 token / 耗时 / 成功率 / 质量分？ - [] 是否有**异常率 / 隔离率 / 覆盖率**报告？ - [] 是否有**隔离/复核队列**可查待审项？#### G. 人机协同 - [] 是否**自动化优先**，仅在模型穷尽失败时转人工？ - [] 人工决策是否**留痕并沉淀**为可复用知识 (specs / KB)？ - [] 是否禁止**批量/自动 signoff** 释放关键隔离 (防绕过闸门)？#### H. 多 Agent 专属 (仅当你用多 Agent 时) - [] 是否存在**有向无环图 (DAG)** + 执行前**强规则校验器** (防循环依赖/孤立节点/重复派发)？ - [] 各 Agent 是否拥有**隔离的上下文**，且通过共享状态而非纸条通信？ - [] 写入共享状态时是否打**版本号 + 时间戳 + 写入者标识** (防幻觉覆写)？ - [] 节点出口失败是否触发**局部自愈 (重试≤N次) → 终止 → 降级**，而非全链崩溃？> **本项目落地设计**：H 节全部条款的 concretization 见 SDK 设计稿 [packages/sdk/docs/13-orchestrator.md] 的「确定性闸门抽象层」
(../packages/sdk/docs/13-orchestrator.md#确定性闸门抽象层deterministic-gate-abstraction)
——DAG 强规则校验器 (‘DeterministicGate’ 三维校验 + ‘WorkflowEngine’ 死锁检测)、隔离上下文 + 共享状态 (‘SharedStateStore’ Blackboard 级, 禁文本传纸条)、写版本号+写入者 (‘version+writer_id’ + CAS)、局部自愈→终止→降级 (‘RetryPolicy’ + ‘ReviewQueue’)。
‘strict=True’ 时全开对标; ‘strict=False’ (默认) 仅韧性层在线。--- ## 四、一句话总结> 真正的工业级 AI 系统，从不建立在“祈祷模型今天不犯错”之上；而是假设模型随时会错、API 随时会挂、网络随时会抖——在极端脆弱前提下，靠强有力工程架构硬拉起一条高可用防线。多 Agent 开发真正的**分水岭**，不在于用了多炫酷的框架，也不在于系统跑起来多智能，而在于各种意外**接踵而至**时，你的系统能否在复杂交互中**自始至终保持稳健**。

Response: 這份《通用防翻車原則與檢查清單》是一份典型的「工業級 AI 系統工程 (AI System Engineering)」指南。它的核心哲學是「零信任 LLM (Zero-Trust LLM)」，即把 LLM 視為一個不可靠、概率性、隨機性強的「提議生成器 (Proposer)」，並在外部用傳統軟體工程（確定性代碼、SSOT、熔斷器、DAG 檢測）建立一個硬實的「防禦性骨架」。

如果將這份原則與 **Agent-to-Agent (A2A) 協議** 進行深度對比，會發現兩者的**目標高度一致**（都是為了消除 AI 的不確定性、防止級聯失效），但**側重點與作用層級不同**：

- **防翻車原則**：關注的是「**系統內部 (In-System)** / 單個 Agent 或閉環多 Agent 系統」的工程防禦機制 (Self-Defensive Architecture)。
- **A2A 協議**：關注的是「**系統之間 (Cross-System)** / 跨組織、跨邊界的 Agent」如何建立標準化、可信任的通信合約 (Interoperability Standard)。

以下從四個關鍵維度，將 A2A 協議的設計與這份防翻車原則進行詳細對比：

1. 情報一致性：共享狀態 (Blackboard) vs. 消息遞送 (JSON-RPC)

比較維度	防翻車原則 (原則 1 & H 節)	A2A 協議	對比分析
數據傳遞	廢除點對點傳紙條，改用「共享狀態庫 (Shared	點對點 (P2P) 消息傳遞，基於 JSON-RPC 2.0 Over	場景不同導致的架構取捨：內部系統有能力維護中央

比較維度	防翻車原則 (原則 1 & H 節)	A2A 協議	對比分析
模式	State Store / 黑板模式)」，寫入時打 version + writer_id 防止覆寫。	HTTPS。	DB；但跨企業的 A2A 無法建立中央黑板，只能走 P2P。
如何防止傳話失真	強制要求數據回寫至 SSOT，LLM 僅作提案者，下遊直接讀取 SSOT。	引入 不可變工件 (Immutable Artifacts) 。中間 Agent 轉發的是原始工件引用/Payload，不允許二次 LLM 摘要。	異曲同工 ：防翻車原則在內部用 DB 鎖定事實；A2A 在跨界傳輸時用「不可變 Artifact」鎖定事實，都杜絕了 LLM 二次加工帶來的幻覺。

2. 邊界與權限：確定性閘門 vs. 能力宣告 (Agent Card)

比較維度	防翻車原則 (原則 2 & 3)	A2A 協議	對比分析
零信任驗證	確定性代碼閘門 (Deterministic Gate) ：格式、事實（出處溯源）、邏輯（業務勾稽，如 $A \approx L + E$ ）三維硬檢測，LLM 不參與裁決。	JSON Schema 約束 (DataPart) + ContentType 協商。若 Schema 不符直接回傳 ContentTypeNotSupportedError。	防翻車原則的邏輯維（勾稽）屬於深度的 業務語意校驗 ；A2A 協議的 Schema 檢查屬於 協議層通信邊界校驗 。兩者是互補的（A2A 負責擋下非法報文，確定性閘門負責擋下邏輯錯誤數據）。
最小權限控制	工具集最小化，不給 Bash/Write，寫操作必須經過 Gate 才可落盤。	Agent Card 能力宣告 + OAuth 2.0/API Key 。Agent 必須顯式宣告其 Capabilities 與 Auth 需求。	防翻車原則控制的是 Agent 的 本地系統調用權限 ；A2A 控制的是 Agent 的 遠端 API 暴露與調用邊界 。

3. 系統韌性與降級：硬上限強殺 vs. 狀態機控制

比較維度	防翻車原則 (原則 5 & H 節)	A2A 協議	對比分析
異常處理機制	熔斷與硬強殺 (Circuit Breaker)：硬時長 (Wall-clock limit)、子進程 subprocess.kill()、DAG 死鎖檢測。	標準化 Task 狀態機：working → input-required / failed / completed。	控制力度的差異：內部系統可以做到 OS 層級的子進程強殺；但 A2A 作為跨網絡協議，無法強殺遠端服務，只能透過 Task 狀態變更與 HTTP 逾時 (Timeout) 來通知 Client 觸發降級。
自愈與兜底	局部自愈 (重試 $\leq N$ 次) → 終止 → 人工復核台 (Human-in-the-loop)。	支持 input-required 狀態，允許 Agent 主動暫停並要求上游/人工補充資訊。	兩者都強調**「不要盲目猜測」**。當資訊不足或失敗時，顯式暴露狀態並引導 Human-in-the-loop 介入。

4. 可觀測性：品質量化看板 vs. 全局 Task 追蹤

比較維度	防翻車原則 (原則 6)	A2A 協議	對比分析
追蹤維度	內部節點 Token 消耗、耗時(ms)、Grounding 率、隔離率/覆蓋率看板。	全局 taskId 與 contextId 貫穿整個傳輸鏈條，支持 SSE 實時狀態流 (Streaming)。	防翻車原則提供的是** APM 級別的內部效能與品質監控**；A2A 提供的是跨系統的分散式 Tracing 基礎標示 (類似 OpenTelemetry Trace ID)。

結論與整合建議

這份防翻車原則寫得非常扎實，代表了單體/微服務 Agent 系統內部的最佳實踐。而 A2A 協議則是 Multi-Agent 走向跨邊界網路化時的通信底座。

如果你要建構一個工業級的 Multi-Agent 系統，最佳的架構是將兩者疊加使用：

- 1. 對內 (In-Process / Intra-Cluster)：嚴格落實防翻車原則。
 - 使用 Shared State Store 替代 Agent 私聊。

- 部署 **三維確定性閘門 (Deterministic Gate)** 攔截 LLM 幻覺。
- 使用 **進程強殺與 DAG 檢測** 防止卡死。

2. 對外 (Inter-System / Cross-Boundary)：將你的 Agent 封裝為 **A2A 規範的端點**。

- 暴露 **Agent Card**，使用 **DataPart (JSON Schema)** 接收外部請求。
- 外部請求進來後，先過 **A2A 協商**，再丟進內部的 **防翻車過濾網** 進行業務勾稽。
- 輸出的產物打包為不可變的 **Artifact** 送回給外部 Agent。

一句话總結：防翻車原則是 Agent 的**「修身（內部工程健壯性）」，**A2A 協議是 Agent 的「齊家（跨系統標準化溝通）」**。**

User prompt: 如果只用A2A，可行嗎

Response: 「只用 A2A，可行，但只能解決通信問題，無法解決系統的可靠性（防翻車）問題。」

如果把 Multi-Agent 系統比喻成一座高鐵網路：

- **A2A 協議是「鐵軌與信號燈標準」**：它規定了列車 (Agent) 之間如何互相通訊、名片格式、如何握手、如何傳遞包裹。
- **防翻車原則是「列車的煞車系統與調度中心」**：它確保列車不會超速、數據不會出軌、遇到故障時能自動熔斷。

如果只用 A2A，而不搭配防翻車工程原則，系統在工業級場景下幾乎必定翻車。

一、只用 A2A 會遇到的四大「工業級死穴」

A2A 是一個**通信協議 (Communication Protocol)**，它的設計目標是「開放性」與「互操作性」，因此它故意不對 Agent 內部的具體實現做硬性約束。如果直接拿 A2A 當作完整的系統架構，會面臨以下問題：

1. 幻覺級聯放大 (Hallucination Cascade)

- **只用 A2A**：Agent A 生成了一個帶有幻覺的 JSON/Text 數據，只要符合 A2A 的 Schema 語法 (Format)，A2A 協議就會順暢地把它透傳給 Agent B。Agent B 基於錯誤數據推導，再透傳給 Agent C，錯誤被指數級放大。
- **缺少了什么**：缺少了防翻車原則中的「**確定性閘門 (Deterministic Gate)**」。必須在 A2A 接收端之後，強制接入代碼級的事實溯源與業務邏輯勾稽（如 $A \approx L + E$ ），LLM 產出的數據才能真正入庫或交付。

2. 无法阻断「传话游戏」的信息衰减

- **只用 A2A**：虽然 A2A 支持 DataPart 和 Artifact，但如果你的上层业务逻辑依然是“Agent A 调用 Agent B，Agent B 总结后再调 Agent C”，A2A 协议无法阻止 LLM 在中间节点自作聪明地“二次加工/摘要”，导致关键上下文丢失。
- **缺少了什么**：缺少了防翻車原則中的「单一可信源（SSOT）与共享状态（Blackboard）」。内部系统必须有一个强类型的共享数据库，Agent 只能向 SSOT 写入版本化数据，下游 Agent 必须直接去 SSOT 读取原始事实，而不是听信上游 Agent 的“口头汇报”。

3. 缺乏底层的「硬熔断」能力

- **只用 A2A**：当远程 Agent 陷入死循环、超时或吐出残缺数据时，A2A 协议层只能等待 HTTP Timeout 或将状态标记为 failed / input-required。它无法强制杀死对方的进程或自动恢复。
- **缺少了什么**：缺少了防翻車原則中的「Circuit Breaker（熔断器）与 OS 级 Hard Cap」。在你的系统内部，必须有独立的子进程隔离、subprocess.kill() 硬超时强杀机制以及优雅降级（退回确定性默认值或人工复核台）。

4. 缺乏交付前对账（Reconciliation）

- **只用 A2A**：A2A 确保了消息“送达”，但无法确保交付给最终用户的报告或结论是“绝对正确”的。
- **缺少了什么**：缺少了交付前的反向核对——逐字逐句校验报告中的每一个数字是否能在 SSOT 中找到元数据引用的 coordinates/page。找不到依据的必须强行删除。

二、什么时候可以“只用 A2A”？

并非所有场景都需要复杂的工程防线。在以下场景中，只依赖 A2A 协议是完全可行且高效的：

1. 轻量级 / 容错率高的协作场景：

- 例如：头脑风暴 Agent + 文案润色 Agent + 翻译 Agent。这种场景下即使有轻微幻觉或描述偏差，也不影响最终使用，不需要严丝合缝的确定性闸门。

2. 纯 API 路由与透传代理：

- 当你的系统只充当 Gateway，负责将 Request 路由给指定的第三方 Agent，并将第三方返回的 SSE 流透传给客户端时，只用 A2A 即可。

3. 生态边界交界处（Perimeter Bridge）：

- 在与外部企业/第三方系统的 Agent 对接时，只用 A2A 作为标准的“接口墙”。

三、工业级系统的分层架构落地方案

真正的工业级落地，不是“二选一”，而是“外部 A2A，内部防翻车”。



总结

- **A2A 解决了“能通”的问题**（解决了跨系统通信规范、能力发现和标准化数据传输）。
- **防翻车原则解决了“可用”的问题**（解决了确定性校验、状态一致性、熔断降级和质量兜底）。

如果只用 **A2A**，你的 Multi-Agent 架构充其量只是“在沙滩上建城堡”——看起来很标准、很先进，但只要大模型一产生严重幻觉或遇到长尾异常，整个系统就会瞬间崩塌。只有把**防翻车原则**作为**内部骨架**，再套上 **A2A 的通信外衣**，才是工业级 AI 应用落地的终极形态。